



Use these Codex prompts to iteratively build the MVP; run and review each output before proceeding.

Assumptions and unspecified items

UNSPECIFIED: Budget, team size (1 founder), launch date, domain/TLD, hosting target, SMS/OTP provider, payment and courier credentials, category taxonomy, fee structure, SLA thresholds, dispute/return policy specifics, and V1 video scope (short-form, capped).

Prompt pack

1. Project folder structure + repo bootstrap

A) Prompt: "Bootstrap a pnpm Turborepo monorepo with Next.js (TS) apps `buyer`, `seller`, `admin`, `api`, plus shared `db/core/ui` packages; include Tailwind, ESLint, Prettier, Docker Compose for Postgres+Redis, and basic health endpoints."

B) Inputs: `NODE_VERSION`, `DATABASE_URL`, `REDIS_URL`, Docker installed.

C) Outputs: Monorepo dirs (`apps/...`, `packages/...`), `docker-compose.yml`, `.env.example`, `turbo.json`, configs.

D) Acceptance: `pnpm -r lint`, `pnpm -r typecheck`, `docker-compose up` succeed; health endpoint returns OK.

E) Snippet:

```
apps/{api,buyer,seller,admin}/
packages/{db,core,ui}/
turbo.json
docker-compose.yml
```

2. Prisma schema + migration + seed

A) Prompt: "Design Prisma schema for marketplace: `User` (with roles), `Seller` (+KYC), `Product` + `Variant` + `Inventory`, `Order` + `OrderEvent`, `Shipment`, `LedgerAccount` + `LedgerEntry`, `PayoutBatch`, `AuditLog`; use `Int` for money (minor units) and include `OrderState` enum; create initial migration and idempotent seed script."

B) Inputs: `DATABASE_URL`; currencies (`SUPPORTED_CURRENCIES=KHR,USD`); unspecified fields (e.g. returnPolicy text).

C) Outputs: `prisma/schema.prisma`, migration files, `seed.ts` (creates admin, sample sellers/products).

D) Acceptance: `pnpm db:migrate` and `pnpm db:seed` run without errors; tables exist; seed is repeatable.

E) Snippet (`OrderState` enum):

```
enum OrderState {
  CREATED
  PAYMENT_PENDING
  PAYMENT_CONFIRMED
  SELLER_CONFIRMED
  PACKED
}
```

```

    IN_TRANSIT
    DELIVERED
    COMPLETED
    CANCELLED
    REFUNDED
  }

```

3. Auth (OTP phone + RBAC)

A) Prompt: "Implement OTP-based login via phone/email (with a UNSPECIFIED provider or dev stub) and role-based access (BUYER, SELLER, ADMIN); issue JWT/session on verify, and add middleware guards for API/routes per role."

B) Inputs: `AUTH_JWT_SECRET`; `OTP_PROVIDER` (UNSPECIFIED); rate-limit config.

C) Outputs: Auth routes (`/api/auth/request-otp`, `/verify-otp`), role guards in `packages/core/auth.ts`, tests.

D) Acceptance: Can't access seller routes as buyer; OTP flow issues tokens; rate limits enforced; `pnpm test` green.

E) Snippet (RBAC check):

```

export type Role = "BUYER" | "SELLER" | "ADMIN";
export function requireRole(user: User, role: Role) {
  if (user.role !== role) throw new Error("FORBIDDEN");
}

```

4. Seller onboarding + KYC + admin approval

A) Prompt: "Build seller onboarding forms (profile, payout bank), KYC doc upload (S3/UNS storage), and admin queue (approve/reject sellers) with statuses; log actions."

B) Inputs: `S3_ENDPOINT`, `S3_BUCKET`, `S3_KEY`, `S3_SECRET`; KYC doc types.

C) Outputs: `Seller`, `SellerDocument` models, upload endpoints, admin approval UI/API, audit logs.

D) Acceptance: Unapproved sellers cannot list products; uploads use signed URLs; admin actions recorded.

E) Snippet (Prisma model):

```

model Seller {
  id          String @id @default(cuid())
  userId      String
  status      String @default("PENDING")
  // ...other fields
}

model SellerDocument {
  id          String @id @default(cuid())
  sellerId    String
  type        String
  s3Key       String
}

```

5. Product catalog + variants + inventory + validation

A) Prompt: "Implement product/variant CRUD with inventory tracking; enforce validation of required fields (price, currency, returns policy, seller contact, images) before listing goes active; include a moderation status."

B) Inputs: Categories (UNSPECIFIED), disclosure templates, seller address format.

C) Outputs: `Product`, `Variant`, `InventoryItem` models, validation logic, endpoints/UI, tests.

D) Acceptance: Missing mandatory fields cause errors; inventory cannot go negative (tests for concurrency).

E) Snippet (validation example):

```
const productSchema = z.object({
  name: z.string().min(3),
  price: z.number().min(0),
  currency: z.enum(["KHR", "USD"]),
  returnPolicy: z.string().min(10),
  sellerContact: z.string().min(5),
});
```

6. Buyer feed + product page + responsive UI

A) Prompt: "Create buyer Next.js (TS) UI with Tailwind: paginated feed API calls, product detail page showing info/variants/seller, and ensure responsive design (mobile/tablet); scaffold locale support (km/en)."

B) Inputs: `DEFAULT_LOCALE=km`, `SUPPORTED_LOCALES=km,en`.

C) Outputs: `apps/buyer` pages/components, fetch hooks, API routes (`/api/products`).

D) Acceptance: Feed loads products; PDP displays details; layout adapts on narrow view; locale switch works.

E) Snippet (product fetch):

```
const { data } = useSWR(`/api/products/${id}`, fetcher);
// Display data.product.title, images, etc.
```

7. Cart + checkout (COD + prepaid stubs)

A) Prompt: "Implement cart and checkout: add/remove items, capture shipping address, calculate totals, then create order (atomic with Prisma tx); support `COD` and `PREPAID` methods with stub PaymentAdapter logic (actual flows in step 8); ensure idempotency."

B) Inputs: `IDEMPOTENCY_TTL=600`; payment methods config.

C) Outputs: `Cart` and `Order` endpoints/UI, reservation logic, idempotency key handling, tests.

D) Acceptance: Race conditions handled (no oversell); same idempotency key returns same order; test concurrency.

E) Snippet (checkout API):

```
await prisma.$transaction(async (tx) => {
  // check stock, decrement inventory, create Order and OrderItems
});
```

8. Payment adapters + idempotent webhooks

A) Prompt: "Create `PaymentAdapter` modules for KHQR (Bakong), ABA PayWay, ACLEDA (Toanchet), and Wing: each with `createPayment`, `verifyWebhook`; implement idempotent webhook routes (e.g. `/api/webhooks/payway`) that log events. Use placeholders if creds/fields UNSPECIFIED."

B) Inputs:

9. KHQR: `BAKONG_BASE_URL`, `BAKONG_ID`, `BAKONG_SECRET`.

10. PayWay: `PAYWAY_BASE_URL`, `PAYWAY_ID`, `PAYWAY_KEY`, `PAYWAY_SECRET`.

11. ACLEDA: `ACLEDA_BASE_URL`, `ACLEDA_ID`, `ACLEDA_KEY`, `ACLEDA_SECRET`.

12. Wing: `WING_BASE_URL`, `WING_ID`, `WING_KEY`, `WING_SECRET`.

C) Outputs: `packages/core/payments/{bakong,payway,acleda,wing}.ts`, webhook endpoints, DB `PaymentEvent`.

D) Acceptance: Webhook replays do not double-process (idempotent); signatures (if set) are validated; unconfigured providers use stub (and tests skip them).

E) Snippet (adapter interface):

```
export interface PaymentAdapter {
  createPayment(orderId: string, amount: number): Promise<string>;
  parseWebhook(req: NextApiRequest): { providerEventId: string; status:
string; orderId?: string };
}
```

13. Strict order state machine + events log

A) Prompt: "Implement strict `OrderState` transition service: define a map of allowed transitions, use it to guard updates, and log each transition to `order_events` (immutable). Include tests for every valid/invalid transition."

B) Inputs: SLA values (UNSPECIFIED).

C) Outputs: `OrderStateMachine.ts`, `OrderEvent` model, unit tests.

D) Acceptance: Invalid transition throws; valid transitions create exactly one `OrderEvent`; test coverage 100%.

E) Snippet (transition guard):

```
const ALLOWED: Record<OrderState, OrderState[]> = {
  CREATED: [ "PAYMENT_CONFIRMED", "CANCELLED" ],
  // ...
};
```

14. Shipping adapter + tracking UI + courier stubs

A) Prompt: "Create a `ShippingAdapter` abstraction: V1 manual entry (select carrier + tracking number), V2 stubbed connectors for J&T, Grab, Cambodia Post; update order state on shipment events; add seller UI to enter tracking, buyer UI to view tracking."

B) Inputs: `CARRIERS=JNT,GRAB,CAMBODIA_POST,OTHER`; API creds UNSPECIFIED.

C) Outputs: `Shipment` / `ShipmentEvent` models, adapter stubs, tracking endpoints, UI components.

D) Acceptance: Entering tracking moves order to `IN_TRANSIT`; buyer sees updates; wrong creds path falls back gracefully.

E) Snippet (Shipment model):

```
model Shipment {
  id          String @id @default(cuid())
  orderId     String @unique
  carrier     String
  trackingNo  String?
}
```

15. Ledger + payout batches + hold windows

A) Prompt: "Implement double-entry ledger for seller balances; after each completed order, credit seller (minus commission) with a hold (based on risk tier); build payout batch job that releases held funds older than tier-specific days; support CSV export for payouts."

B) Inputs: COMMISSION_RATE=0.05 ; HOLD_DAYS_TIER_LOW/MED/HIGH ; seller risk in DB.

C) Outputs: LedgerAccount / LedgerEntry models, payout service, export script, tests.

D) Acceptance: Ledger entries always balance; payouts match release criteria; COD path (cash-collected) doesn't create ghost funds.

E) Snippet (credit ledger):

```
await prisma.ledgerEntry.createMany({
  data: [
    { accountId: `seller:${id}`, amount: orderAmt - fee, type:
'CREDIT' },
    { accountId: 'platform:fee', amount: fee, type: 'CREDIT' },
  ],
});
```

16. Admin/moderation console + disputes + audit log

A) Prompt: "Build admin console: approve/reject sellers and products, handle flagged content, process disputes (orders/refunds) and log all actions to AuditLog; enforce admin role for these pages."

B) Inputs: Admin list (seed or env); dispute reasons (UNSPECIFIED).

C) Outputs: AuditLog model, admin pages (approvals, disputes), API endpoints, tests.

D) Acceptance: Only admins access; each admin action inserts an AuditLog entry; dispute workflow moves orders to REFUNDED when approved.

E) Snippet (AuditLog model):

```
model AuditLog {
  id          String @id @default(cuid())
  actorId     String
  action      String
  entity      String
  entityId    String
  timestamp   DateTime @default(now())
}
```

17. Seed data script (admin, demo sellers, products)

A) Prompt: "Write an idempotent seed script creating one admin user, two active sellers, a pending seller, and ~20 products with bilingual fields (Khmer/English), categories, and a few sample orders."

B) Inputs: Sample data (names, categories UNSPECIFIED); `SEED_ENABLED` flag.

C) Outputs: `seed.ts` in `packages/db` or `scripts/`, executed via `pnpm db:seed`.

D) Acceptance: Re-running seed doesn't duplicate; dev UI shows seeded data; basic API calls return seeded entries.

E) Snippet (seed admin):

```
await prisma.user.upsert({
  where: { email: "admin@example.com" },
  create: { email: "admin@example.com", role: "ADMIN" },
  update: {},
});
```

18. Integration tests (checkout, webhooks, transitions)

A) Prompt: "Add integration tests (Vitest) using Testcontainers for Postgres/Redis: cover checkout under concurrency, webhook idempotency, order state transitions, and ledger reconciliation end-to-end; include CI config."

B) Inputs: `CI=true`; test DB URLs (auto-set by containers).

C) Outputs: `tests/integration/*.test.ts`, Docker Testcontainers setup, CI pipeline YAML.

D) Acceptance: Tests pass consistently; simulate double-spend checkout and webhook replay (no duplicate); CI passes on new branch.

E) Snippet (webhook idempotency test):

```
await sendWebhook(eventPayload);
await sendWebhook(eventPayload);
const count = await prisma.paymentEvent.count({ where: {
  idempotencyKey } });
expect(count).toBe(1);
```

19. docker-compose + dev scripts + migrations + CI

A) Prompt: "Create `docker-compose.yml` for Postgres+Redis, `.env.example`, `README` instructions (migrate, seed, dev), and a CI workflow (GitHub Actions) to lint, type-check, migrate a test DB, and run all tests."

B) Inputs: `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `REDIS_URL` as examples.

C) Outputs: `docker-compose.yml`, `.env.example`, `README.md`, `.github/workflows/ci.yml`.

D) Acceptance: `docker compose up` + `pnpm dev` runs without manual setup; CI passes on build.

E) Snippet (docker-compose):

```
version: '3.8'
services:
  db:
```

```
image: postgres:16
environment:
  POSTGRES_USER: postgres
  POSTGRES_DB: marketplace
redis:
  image: redis:7
```

Reference vendor docs (Cambodia)

```
Bakong KHQR      https://bakong.nbc.gov.kh/en/
ABA PayWay       https://developer.payway.com.kh/
ACLEDA Toanchet (sandbox) https://toanchetpay.acledabank.com.kh/
toanchetpay/sandbox
Wing Merchant    https://www.wingbank.com.kh/en/business/payments-collections/
wing-merchant
J&T API          https://developer.jet.co.id/
GrabExpress Biz (KH) https://www.grab.com/kh/en/business/express/
Cambodia Post    https://cambodiapost.com.kh/home
```
